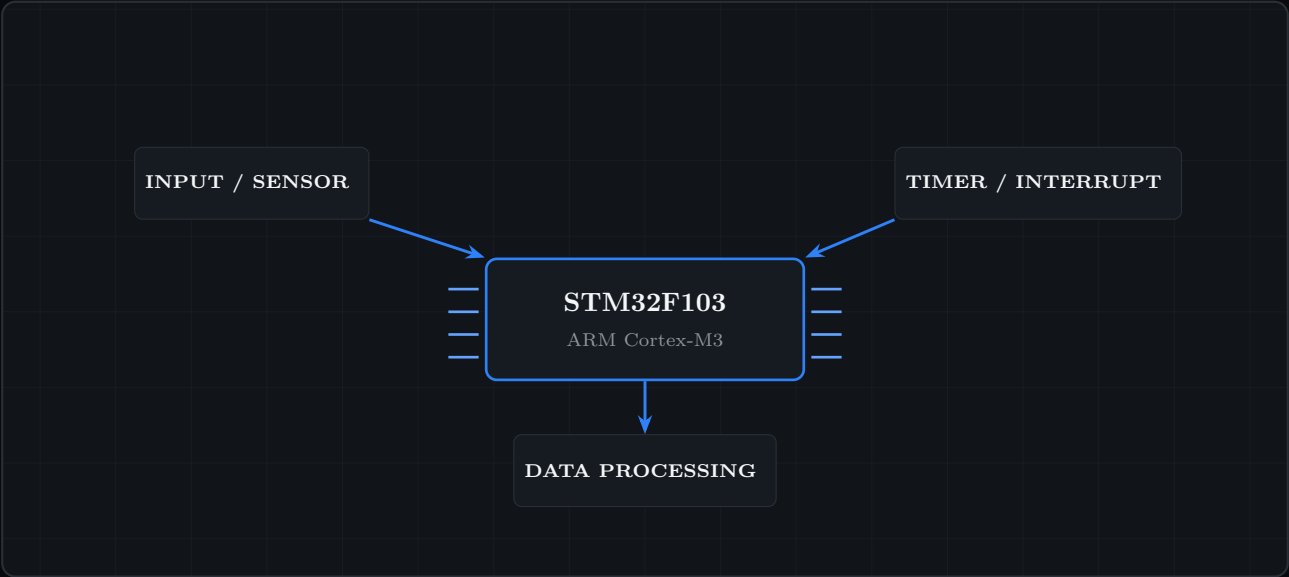


STM32 Bare-Metal Acquisition System

Embedded Systems · ARM Cortex-M3 · C · Register-Level Programming



STM32F103 · ARM Cortex-M3 · Bare-Metal C · Interrupts · Timers

Understanding the MCU below the abstraction layer

PROJECT CORE

MCU

STM32F103

CPU

ARM Cortex-M3

Language

C

Approach

Bare-metal

Why bare-metal?

Bare-metal development removes the abstraction layer normally provided by high-level frameworks and requires direct interaction with the microcontroller architecture and peripherals.

For an STM32-based acquisition system, this means configuring the required hardware resources, controlling registers, handling timing, and designing the execution flow directly in C.

Engineering objective

The project was developed around an STM32F103 microcontroller and focused on embedded acquisition, peripheral configuration, timing, and interrupt-driven execution.

Core engineering questions

- How does the CPU configure a peripheral?
- How are GPIO resources initialized at register level?
- How can acquisition be synchronized with timing?
- How can interrupts react to hardware events?
- How does the firmware structure data acquisition deterministically?

KEY IDEA

Direct hardware control

The firmware interacts directly with MCU resources instead of relying on a high-level framework.

ENGINEERING STACK

ARM Cortex-M3

CPU architecture

Bare-Metal C

Low-level firmware

Peripheral Control

Registers / GPIO

Real-Time Events

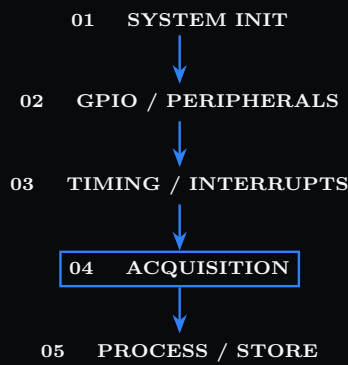
Timers / interrupts

The project connects low-level firmware design with deterministic hardware interaction and data acquisition.

Inside the acquisition pipeline

Execution model

The firmware can be understood as a sequence of hardware initialization, acquisition, timing, processing, and response operations.



Hardware / software boundary

A bare-metal system exposes the relationship between firmware and microcontroller hardware. Configuration registers define how the peripherals behave; firmware then orchestrates their operation.



DESIGN PRINCIPLE

Configure → Acquire → React → Process

The firmware is structured around explicit hardware states and deterministic execution paths.

SYSTEM VIEW

Physical signal → Peripheral → Interrupt / Timing → Firmware Logic

Programming below the abstraction layer

Register-level GPIO configuration

A central part of bare-metal development is understanding that peripherals are controlled through memory-mapped registers. The following example illustrates the structure of a direct GPIO configuration on an STM32F1 device.

```

1  #define RCC_APB2ENR (*(volatile uint32_t*)0x40021018)
2  #define GPIOA_CRL (*(volatile uint32_t*)0x40010800)
3
4  void gpio_init(void)
5  {
6      /* Enable GPIOA clock */
7      RCC_APB2ENR |= (1U << 2);
8
9      /* Configure PA0 as input */
10     GPIOA_CRL &= ~(0xFU << 0);
11     GPIOA_CRL |= (0x4U << 0);
12 }

```

Representative bare-metal implementation showing direct register access.

Timer-driven execution

Timing is fundamental to acquisition systems. A hardware timer can provide a deterministic time base and generate an interrupt when a programmed period expires.

```

1  volatile uint8_t sample_flag = 0;
2
3  void TIM2_IRQHandler(void)
4  {
5      /* Clear update interrupt */
6      TIM2->SR &= ~(1U << 0);
7      sample_flag = 1;
8  }
9
10 int main(void)
11 {
12     system_init();
13     timer_init();
14
15     while (1) {
16         if (sample_flag) {
17             sample_flag = 0;
18             acquire_sample();
19         }
20     }
21 }

```

Representative interrupt-driven acquisition pattern.

WHY THIS MATTERS

Registers are not just implementation details — they define the hardware.

Working at this level develops a practical understanding of clocks, memory-mapped peripherals, bit manipulation, interrupt sources, and the relationship between software instructions and hardware behavior.

GPIO

Digital hardware control

TIMERS

Deterministic timing

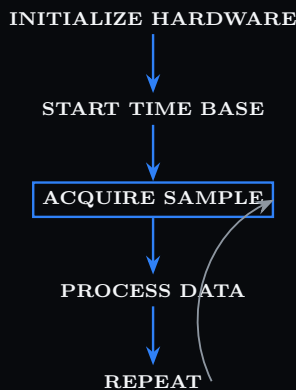
INTERRUPTS

Event-driven execution

Building a deterministic acquisition loop

Acquisition sequence

The firmware can be represented as a deterministic acquisition chain: hardware initialization establishes the system state, a timing source defines when acquisition occurs, and the main execution path processes the resulting data.



Acquisition logic

```

1  static volatile uint16_t sample;
2
3  uint16_t acquire_sample(void)
4  {
5      /* Start / trigger ADC conversion */
6      ADC1->CR2 |= (1U << 22);
7
8      /* Wait for conversion complete */
9      while (!(ADC1->SR & (1U << 1)))
10         ;
11
12     return (uint16_t)ADC1->DR;
13 }
14
15 void process_sample(uint16_t value)
16 {
17     /* Example processing stage */
18     sample = value;
19 }
  
```

Representative STM32F1 ADC acquisition pattern. The exact register configuration should be adapted to the original implementation.

EXECUTION MODEL

Hardware event

Timer / interrupt defines the acquisition opportunity.

Acquisition

Peripheral captures the physical signal.

Processing

Firmware handles the acquired value.

DESIGN TAKEAWAY

The acquisition loop separates timing, hardware capture, and software processing — a foundation for reliable embedded systems.

What this project proves

This project demonstrates a deeper level of embedded-system understanding: not only using a microcontroller, but understanding how firmware configures and controls the underlying hardware.

01

Bare-Metal C

Direct interaction with MCU resources and low-level firmware structures.

02

ARM Cortex-M3

Practical understanding of a Cortex-M embedded execution environment.

03

Peripheral Control

GPIO, timing and acquisition resources configured at low level.

04

Interrupt-Driven Design

Hardware events used to structure deterministic firmware execution.

Registers · Peripherals · Timing · Interrupts · Acquisition ;